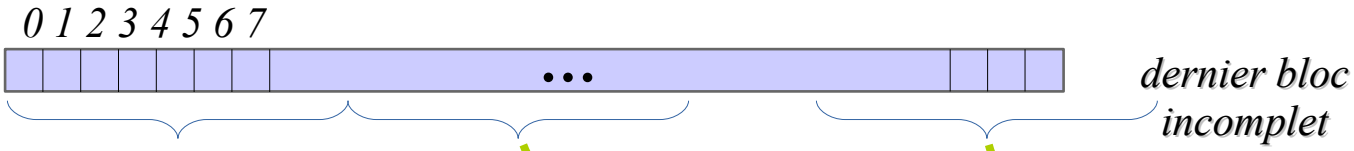
Systèmes et Réseaux 1

Partie *Systèmes d'exploitation*

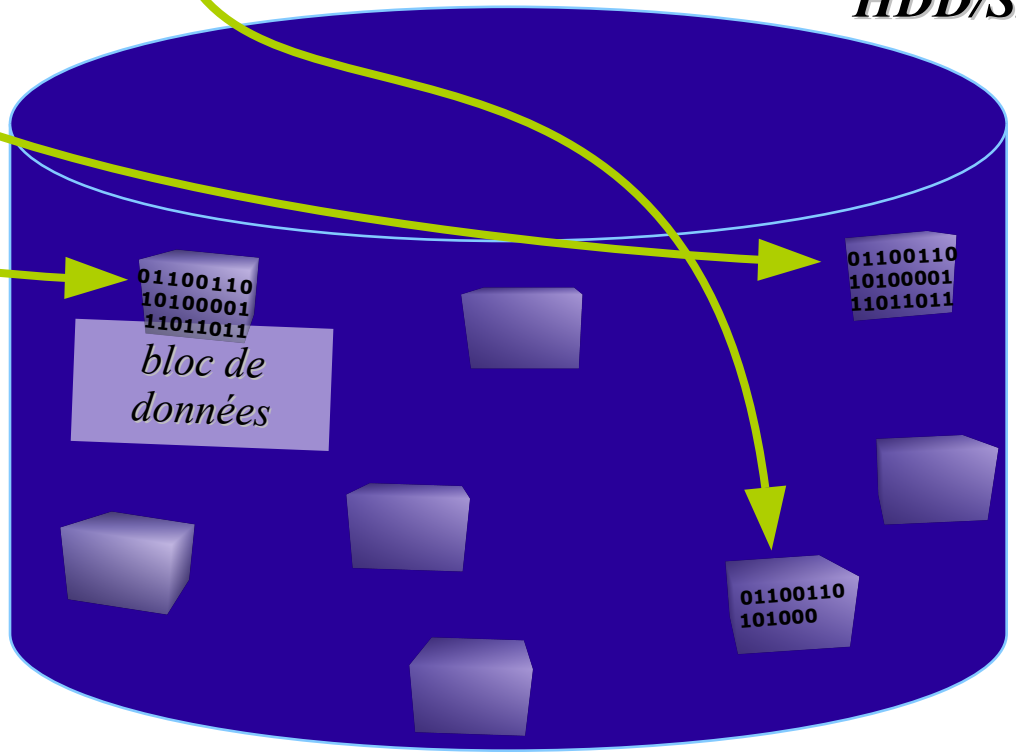
4 – Structure interne du système de fichiers Unix

Système de Gestion de Fichiers Unix : introduction



bloc de 512 octets

HDD/SSD



Fichier = suite d'octets

Système de Gestion de Fichiers Unix : introduction

Rappels :

- un fichier est une suite finie d'octets. Il est matérialisé par des blocs du disque
- le Système de Gestion de Fichiers (SGF) s'occupe de faire le lien entre les données enregistrées dans les blocs du disque et la représentation des informations sous forme de fichiers (fichiers ordinaires, répertoires, liens...) et leur désignation
- il utilise des structures de données enregistrées dans le Système de Fichiers (SF) , notamment la structure de base du SF et les *inodes*

SGF – Primitives Unix : Accès aux caractéristiques d'un fichier

- accès aux informations contenues par l'*inode* d'un fichier :

```
#include <sys/types.h>
#include <sys/stat.h>
int   stat  (const char * NomFich, struct stat * pInfos)
int   fstat (int  DescFich, struct stat * pInfos)
int   lstat (const char * NomFich, struct stat * pInfos)
```

- copie les informations de l'*inode* du fichier dans la structure struct stat dont l'adresse est passée en paramètre (pInfos)
- stat prend en paramètre la désignation du fichier NomFich
- fstat prend en paramètre le descripteur du fichier DescFich
  (fichier doit avoir été ouvert)
- lstat contrairement à stat, ne suit pas les liens symboliques
  (renvoie le lien plutôt que le fichier lié)
- Ces fonctions retournent -1 en cas de problème ou 0 sinon.

SGF – Primitives Unix : Accès aux caractéristiques d'un fichier

- La structure **struct stat** contient *(au moins)* les champs suivants :

```
struct stat
{
  dev_t    st_dev;    /* Id du système de fichiers contenant le fichier */
  ino_t    st_ino;    /* N° de l'inode du fichier */
  mode_t   st_mode;   /* Type et mode du fichier */
  nlink_t  st_nlink;  /* Nb de liens physiques sur le fichier */
  uid_t    st_uid;    /* Id de l'utilisateur·ise propriétaire du fichier */
  gid_t    st_gid;    /* Id du groupe de l'utilisateur·ise propriétaire du fichier */
  off_t    st_size;   /* Taille du fichier en octets */
  time_t   st_atime;  /* Instant du dernier accès au fichier */
                      /* (création, lecture, écriture, lecture des caractéristiques,...) */
  time_t   st_mtime;  /* Instant de la dernière modification des données */
  time_t   st_ctime;  /* Instant de la dernière modification des caractéristiques */
                      /* (création, écriture, changement des droits, changement de propriétaire,...)
};
```

SGF – Primitives Unix : Accès aux caractéristiques d'un fichier

- le champ st_mode contient plusieurs informations dont le type du fichier et ses droits d'accès.
- Pour obtenir le type du fichier, on fait appel aux macros fonctions suivantes (on suppose que Infos est une variable de type struct stat) :
  - S_ISREG(Infos.st_mode) ≠ 0 si c'est un fichier ordinaire,
  - S_ISDIR (Infos.st_mode) ≠ 0 si c'est un répertoire,
  - S_ISFIFO(Infos.st_mode) ≠ 0 si c'est un fichier spécial tube *(FIFO)*,
  - S_ISCHR(Infos.st_mode) ≠ 0 si c'est un périphérique caractère,
  - S_ISBLK (Infos.st_mode) ≠ 0 si c'est un périphérique bloc,
  - S_ISLNK (Infos.st_mode) ≠ 0 si c'est un lien symbolique,
  - S_ISSOCK(Infos.st_mode) ≠ 0 si c'est un *socket* (connexion réseau).

SGF – Primitives Unix : Accès aux caractéristiques d'un fichier

- Ou bien utiliser les macros constantes suivantes par exemple pour déterminer tous les types dans une structure switch
  (on suppose que Infos est une variable de type struct stat) :

```
#include <fcntl.h>
switch (Infos.st_mode & S_IFMT) {     /* S_IFMT masque binaire du type de fichier */
  case  S_IFREG:     printf("fichier ordinaire\n");              break;
  case  S_IFDIR:     printf("répertoire\n");                     break;
  case  S_IFIFO:     printf("fichier spécial tube (FIFO)\n");    break;
  case  S_IFCHR:     printf("périphérique caractère\n");         break;
  case  S_IFBLK:     printf("périphérique bloc\n");              break;
  case  S_IFLNK:     printf("lien symbolique\n");                break;
  case  S_IFSOCK:    printf("socket\n");                         break;
  default:           printf("inconnu ?\n");                      break;
}
```

SGF – Primitives Unix : Accès aux caractéristiques d'un fichier

- Pour obtenir les droits d'accès dans st_mode, on utilise l'opérateur & et les constantes correspondantes (vues avec la primitive open) comme masques binaires. Par exemple :
  (Infos.st_mode & S_IRUSR) ≠ 0 si le/la propriétaire a le droit d'accès en lecture

- Les champs st_atime, st_mtime et st_ctime contiennent des instants (dates et heures) sous la forme d'entiers (nombre de secondes écoulées depuis le 01/01/1970, c'est-à-dire le « big-bang » Unix).

- Pour les afficher sous une forme plus lisible, utiliser la fonction standard ctime :

```
#include <time.h>
char * ctime(const time_t * pTemps)
```

Exemple : printf("%s", ctime(&Infos.st_atime));

SGF – Primitives Unix : Accès aux caractéristiques d'un fichier

- Exercice 4.1

  écrire un programme C qui prend en argument la désignation d'un fichier et affiche les caractéristiques suivantes : désignation du fichier, taille, date de dernière modification des données.

SGF – Primitives Unix : Accès aux caractéristiques d'un fichier

Instants en nanosecondes...

- SE & SF modernes permettent d'enregistrer les instants (date & heure) avec une précision à la nanoseconde.
- La structure struct stat remplie par une des fonctions stat, contient les champs st_atim, st_mtim et st_ctim de type struct timespec

```
#include <time.h>
struct timespec
{
   time_t  tv_sec;   /* secondes */
   long    tv_nsec; /* nanosecondes */
};
```

- On peut toujours afficher date et heure avec ctime :

```
printf("%s", ctime(&Infos.st_atim.tv_sec));
```

SGF – Accès aux répertoires

- Un répertoire est un fichier qui contient le nom des éléments contenus dans ce répertoire ainsi que leur *inode*.
- Par exemple, le contenu d'un répertoire contenant les fichiers fich1.txt et fich2.txt et le répertoire REP peut être représenté par la table suivante :

les éléments, pas dans l'ordre

Nom	N° *inode*	
.	8186182	*le répertoire lui-même*
fich2.txt	8186327	
..	1784306	*le répertoire au-dessus*
REP	8333283	
fich1.txt	8186211	

SGF – Parcours d'un répertoire

Ouverture et lecture d'un répertoire :

```
#include <sys/types.h>
#include <dirent.h>
DIR * opendir(const char * DesignationRep)
```

- ouvre le répertoire de désignation DesignationRep
- retourne son identificateur ou NULL en cas de problème

```
struct dirent * readdir(DIR * Rep)
```

- lit la structure qui représente l'élément courant, retourne un pointeur vers celle-ci, et passe à l'élément suivant
- retourne NULL en cas de problème ou si la fin du répertoire est atteinte
- struct dirent peut être statique dans la fonction readdir *(ne pas faire de* free *!)* et réécrite à la lecture suivante

SGF – Parcours d'un répertoire

- Lecture d'un répertoire :

```
#include <sys/types.h>
#include <dirent.h>
struct dirent * readdir(DIR * Rep)
```

- La structure struct dirent contient au moins les champs :

char d_name[]; → chaîne de caractères (terminée par \0) contenant le nom de l'élément (fichier ordinaire, répertoire...)

ino_t d_ino; → numéro de *inode* (entier long)

- d'autres champs existent mais ne sont pas standards et pas présents sur tous les SF

Attention : la fonction readdir peut ou non, selon les implémentations, retourner les éléments de nom "." et ".."

SGF – Parcours d'un répertoire

- *Rembobinage* et fermeture d'un répertoire :

```
#include <sys/types.h>
#include <dirent.h>

void  rewinddir(DIR * Rep)
```

- remet la position courante au début du répertoire d'identificateur Rep.

```
int     closedir(DIR * Rep)
```

- ferme le répertoire d'identificateur Rep
- retourne -1 en cas de problème ou 0 sinon

SGF – Parcours d'un répertoire

- Exercice 4.2

écrire une fonction d'en-tête :

```
long  TailleRepCour(void)
```

qui retourne le nombre total d'octets occupés par les fichiers ordinaires présents dans le répertoire courant. Attention à ne pas suivre les liens symboliques.

La fonction retourne -1 en cas de problème.

SGF – Parcours d'une sous-arborescence

- La fonction de parcours d'une sous-arborescence de racine Rep doit être récursive et suivre l'algorithme général suivant :

```
Parcourir(Rep)
{
   Pour tout Element de Rep
      Si Element est différent de "." et de ".."
         Si Rep/Element est un répertoire
            Parcourir(Rep/Element);
         Sinon
            Traiter(Rep/Element);
}
```

SGF – Parcours d'une sous-arborescence

- L'algorithme précédent nécessite de construire des désignations, c-à-d de manipuler des chaînes de caractères.

  → fonctions de manipulation des chaînes de la bibliothèque standard :

```
#include <string.h>

char * strcpy(char * dest,  const char * src);    /* copie de chaîne */
```

- copie la chaîne de caractères de `src` vers `dest` , qui doit être assez longue pour contenir toute la chaîne (y compris \0 à la fin)

```
char * strcat(char * dest,  const char * src);    /* concaténation de chaînes */
```

- copie la chaîne de caractères de `src` après la chaîne déjà présente dans `dest`, qui doit être assez longue pour contenir les deux chaînes (y compris \0 à la fin)

SGF – Parcours d'une sous-arborescence

- Exemple :

```
#include <string.h>
...
char ch1[512],ch2[512];
strcpy(ch1,"Il fait ");   /* ch1 contient "Il fait " */
strcpy(ch2,ch1);          /* ch1 contient "Il fait " et ch2 contient "Il fait " */
strcpy(ch1,"beau");       /* ch1 contient "beau" et ch2 contient "Il fait " */
strcat(ch2,ch1);          /* ch2 contient "Il fait beau" et ch1 contient "beau" */
```

enfin nous aurons besoin de comparer deux chaîne de caractères :

```
int  strcmp(const char * s1,  const char * s2);
```

- compare les deux chaînes de caractères s1 et s2
- retourne : 0 quand les deux chaînes sont identiques (s1 = s2)

  une valeur < 0 si les chaînes sont dans l'ordre lexicographique (s1 < s2)

  une valeur > 0 si les chaînes sont dans l'ordre inverse (s1 > s2)

SGF – Parcours d'une sous-arborescence

- Alternative pour des copies plus complexes :

```
#include <stdio.h>

int  snprintf(char * str,  size_t  size,  const char * format, ...);
```

- fonctionne comme printf excepté que la chaîne générée est copiée dans str et la taille de la chaîne est au plus size octets (y compris \0 à la fin)

- str doit correspondre à une chaîne allouée ou statique, dont la taille est (au moins) size octets

Sous Linux, taille max d'une désignation : 4096 octets (getconf -a | grep PATH_MAX)

SGF – Parcours d'une sous-arborescence

- Exercice 4.3
  écrire une fonction qui affiche à l'écran les désignations de tous les fichiers ordinaires contenus dans la sous-arborescence dont le répertoire racine est passé en paramètre.

- Par exemple, supposons que l'on se trouve dans la configuration suivante :
  - le répertoire TEST contient le fichier fich.txt, les répertoires REP1 et REP2 ;
  - le répertoire REP1 contient les fichiers fich11.txt et fich12.txt ;
  - le répertoire REP2 contient le fichier fich2.txt.

- Si le répertoire TEST est passé en paramètre à la fonction, elle doit alors afficher :
  TEST/REP1/fich11.txt
  TEST/REP1/fich12.txt
  TEST/REP2/fich2.txt
  TEST/fich.txt